In the ELBO, the KL term can be computed in closed-form, as both $\mathrm{q}(\mathbf{u}_d^l)$ and $\mathrm{p}(\mathbf{u}_d^l)$ are Gaussians. The log likelihood term can be approximated by sampling from the marginal posterior $\mathrm{q}(\mathbf{f}_{n,:}^L)$, which can be done efficiently through univariate Gaussians as in (Salimbeni & Deisenroth, 2017). Specifically, $\mathbf{U}^l$ can be analytically marginalized in eq. (3), which yields $\mathrm{q}(\{\mathbf{F}^l\}^l) = \prod_l \mathrm{q}(\mathbf{F}^l|\mathbf{F}^{l-1}, \mathbf{W}^l) = \prod_{l,d} \mathcal{N}(\mathbf{f}_d^l|\tilde{\boldsymbol{\mu}}_d^l, \tilde{\boldsymbol{\Sigma}}_d^l)$, with:

$$[\tilde{\boldsymbol{\mu}}_d^l]_i = \mu_d^l(a_{id}^l) + \boldsymbol{\alpha}_d^l(a_{id}^l)^\intercal(\mathbf{m}_d^l - \mu_d^l(\mathbf{z}_d^l)), \tag{5}$$

$$[\tilde{\boldsymbol{\Sigma}}_d^l]_{ij} = k_d^l(a_{id}^l, a_{jd}^l) - \boldsymbol{\alpha}_d^l(a_{id}^l)^\intercal(k_d^l(\mathbf{z}_d^l) - \mathbf{S}_d^l)\boldsymbol{\alpha}_d^l(a_{jd}^l), \tag{6}$$

where $\boldsymbol{\alpha}_d^l(x) = k_d^l(x, \mathbf{z}_d^l)[k_d^l(\mathbf{z}_d^l)]^{-1}$ and $\mathbf{a}_{n,:}^l = \mathbf{W}^l\mathbf{f}_{n,:}^{l-1}$. Importantly, the marginal posterior $\mathrm{q}(\mathbf{f}_{n,:}^l)$ is a Gaussian that depends only on $\mathbf{a}_{n,:}^l$, which in turn only depends on $\mathrm{q}(\mathbf{f}_{n,:}^{l-1})$. Therefore, sampling from $\mathbf{f}_{n,:}^l$ is straightforward using the reparametrization trick (Kingma & Welling, 2013):

$$f_{nd}^l = [\tilde{\boldsymbol{\mu}}_d^l]_n + \varepsilon \cdot [\tilde{\boldsymbol{\Sigma}}_d^l]_{nn}^{1/2}, \quad \text{with } \varepsilon \sim \mathcal{N}(0,1), \quad \text{and } \mathbf{f}_{n,:}^0 = \mathbf{x}_{n,:}. \tag{7}$$

Training consists in maximizing the ELBO, eq. (4), w.r.t. variational parameters $\{\mathbf{m}_d^l, \mathbf{S}_d^l\}$, inducing points $\{\mathbf{z}_d^l\}$, and model parameters (i.e. weights $\{\mathbf{w}_d^l\}$ and kernel parameters $\{\boldsymbol{\theta}_d^l\}$). This can be done in batches, allowing for scalability to very large datasets. The complexity to evaluate the ELBO is $\mathcal{O}(NM^2(D^1 + \cdots + D^L))$, the same as DGPs with DSVI (Salimbeni & Deisenroth, 2017).[3]

Predictions. Given a new $\mathbf{x}_{*,:}$, we want to compute[4] $\mathrm{p}(\mathbf{f}_{*,:}^L|\mathbf{X}, \mathbf{Y}) \approx \mathbb{E}_{\mathrm{q}(\{\mathbf{U}^l\})}\left[\mathrm{p}(\mathbf{f}_{*,:}^L|\{\mathbf{U}^l\})\right]$. As in (Salimbeni & Deisenroth, 2017), this can be approximated by sampling S values up to the $(L-1)$-th layer with the same eq. (7), but starting with $\mathbf{x}_{*,:}$. Then, $\mathrm{p}(\mathbf{f}_{*,:}^L|\mathbf{X}, \mathbf{Y})$ is given by the mixture of the S Gaussians distributions obtained from eqs. (5)-(6).

Triangular kernel. One of the most popular kernels in GPs is the RBF (Williams & Rasmussen, 2006), which produces very smooth functions. However, the ReLu non-linearity led to a general boost in performance in DNNs (Nair & Hinton, 2010; Glorot et al., 2011), and we aim to model similar activations. Therefore, we introduce the use of the *triangular* (TRI) kernel. Just like RBF, TRI is an isotropic kernel, i.e. it depends on the distance between the inputs, $k(x, y) = \gamma \cdot g(|x - y|/\ell)$, with γ and ℓ the amplitude and lengthscale. For RBF, $g(t) = e^{-t^2/2}$. For TRI, $g(t) = \max(1 - t, 0)$. This is a valid kernel (Williams & Rasmussen, 2006, Section 4.2.1). Similarly to the ReLu, the functions modelled by TRI are piecewise linear, see Figure 6a in the main text and Figure 8 in Appendix C.

Comparison with DGP. The difference between auNN and DGP units is graphically illustrated in Figure 2a. Whereas DGP mappings from one layer to the next are complex functions defined on D^{l-1} dimensions ($D^{l-1} = 2$ in the figure), auNN mappings are defined just along one direction via the weight projection. This is closer in spirit to NNs, whose mappings are also simpler and better suited for feature extraction and learning more abstract concepts. Moreover, since the GP is defined on a 1D space, auNN requires fewer inducing points than DGP (which, intuitively, can be interpreted as inducing (hyper)planes in the D^{l-1}-dimensional space before the projection).

3 Experiments

In this section, auNN is compared to BNN, fBNN (Sun et al., 2019) and DSVI DGP (Salimbeni & Deisenroth, 2017). BNNs are trained with BBP (Blundell et al., 2015), since auNN also leverages a simple VI-based inference approach. In each section we will highlight the most relevant experimental aspects, and all the details can be found in Appendix B. In the sequel, NLL stands for Negative Log Likelihood. Anonymized code for auNN is provided in the supplementary material, along with a script to run it for the 1D illustrative example of Section 3.1.

3.1 An illustrative example

Here we illustrate the two aspects that were highlighted in the introduction: the underestimation of predictive uncertainty for instances located in-between two clusters of training points and the

[3] As in (Salimbeni & Deisenroth, 2017), there exists also a cubic term $\mathcal{O}(M^3(D^1 + \cdots + D^L))$ that is dominated by the former (since the batch size N is typically larger than M). Moreover, in auNN we have the multiplication by weights, with complexity $\mathcal{O}(ND^{l-1}D^l)$ for each layer. This is also dominated by the former.

[4] The distribution $\mathrm{p}(\mathbf{y}_{*,:}^L|\mathbf{X}, \mathbf{Y})$ is obtained as the expectation of the likelihood over $\mathrm{p}(\mathbf{f}_{*,:}^L|\mathbf{X}, \mathbf{Y})$. A Gaussian likelihood is used for regression, and the Robust-Max (Hernández-Lobato et al., 2011) for classification.

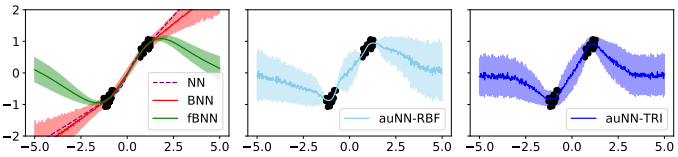

Figure 3: Predictive distribution (mean and one standard deviation) after training on a 1D dataset with two clusters of points. This simple example illustrates the main limitations of NN, BNN and fBNN, which are overcome by the novel auNN. See Table 1 for a summary and the text for details.

Table 1: Visual overview of conclusions from the 1D experiment in Figure 3. This shows that NN, BNN, fBNN and auNN increasingly expand their capabilities.

	Epistemic uncertainty	Reverts to the mean	In-between uncertainty
NN	✗	✗	✗
BNN	✓	✗	✗
fBNN	✓	✓	✗
auNN	✓	✓	✓

extrapolation to OOD data. Figure 3 shows the predictive distribution of NN, BNN, fBNN and auNN (with RBF and TRI kernels) after training on a simple 1D dataset with two clusters of points. All the methods have one hidden layer with 25 units, and 5 inducing points are used for auNN.

In Figure 3, the deterministic nature of NNs prevents them from providing epistemic uncertainty (i.e. the one originating from the model (Kendall & Gal, 2017)). Moreover, there is no prior to guide the extrapolation to OOD data. BNNs provide epistemic uncertainty. However, the prior in the complex space of weights does not allow for guiding the extrapolation to OOD data (e.g. by reverting to the empirical mean). Moreover, note that BNNs underestimate the predictive uncertainty in the region between the two clusters, where there is no observed data (this region is usually called the *gap*). More specifically, as shown in (Foong et al., 2020), the predictive uncertainty for data points in the gap is limited by that on the extremes. By specifying the prior in function space, fBNN can induce properties in the output, such as reverting to the empirical mean for OOD data through a zero-mean GP prior. However, the underestimation of in-between uncertainty persists, since the posterior stochastic process for fBNN is based on a weight-space factorized Gaussian (as BNN with BBP), see (Sun et al., 2019, Section 3.1) for details. Finally, auNN (either with RBF or TRI kernel) addresses both aspects through the novel activation-level modelling of uncertainty, which utilizes a zero-mean GP prior for the activations. Table 1 summarizes the main characteristics of each method. Next, a more comprehensive experiment with deeper architectures and more complex multidimensional datasets is provided.

3.2 UCI REGRESSION DATASETS WITH GAP SPLITS

Standard splits are not appropriate to evaluate the quality of uncertainty estimates for in-between data, since both train and test sets may cover the space equally. This motivated the introduction of gap splits (Foong et al., 2019). Namely, a set with D dimensions admits D such train-test partitions by considering each dimension, sorting the points according to its value, and selecting the middle 1/3 for test (and the outer 2/3 for training), see Figure 2c. With these partitions, overconfident predictions for data points in the gap manifest as very high values of test negative log likelihood.

Using the gap splits, it was recently shown that BNNs yield overconfident predictions for in-between data (Foong et al., 2019). The authors highlight the case of Energy and Naval datasets, where BNNs fail catastrophically. Figure 4a reproduces these results for BNNs and checks that fBNNs also obtain overconfident predictions, as theoretically expected. However, notice that activation-level stochasticity performs better, specially through the triangular kernel, which dramatically improves the results (see the plot scale). Figure 4b confirms that the difference is due to the underestimation of uncertainty, since the predictive performance in terms of RMSE is on a similar scale for all the methods. In all cases, $D = 50$ hidden units are used, and auNN uses $M = 10$ inducing points.

To further understand the intuition behind the different results, Figure 5 shows the predictive distribution over a segment that crosses the gap, recall Figure 2c. We observe that activation-level approaches obtain more sensitive (less confident) uncertainties in the gap, where there is no observed data. For instance, BNN and fBNN predictions in Naval are unjustifiably overconfident, since the output in that dataset ranges from 0.95 to 1. Also, to illustrate the internal mechanism of auNN, Figure 6a shows one example of the activations learned when using each kernel. Although it is just one example, it allows for visualising the different nature: smoother for RBF and piecewise linear for TRI. All the activations for a particular network and for both kernels are shown in Appendix C (Figure 8).

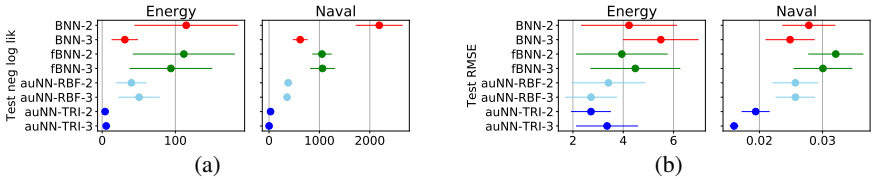

Figure 4: Test NLL (a) and RMSE (b) for the gap splits in Energy and Naval datasets (mean and one standard error, the lower the better). Activation-level uncertainty, specially through the triangular kernel, avoids the dramatic failure of BNN and fBNN in terms of NLL (see the scale). The similar values in RMSE reveal that this failure actually comes from an extremely overconfident estimation by BNN and fBNN, see also Figure 5.

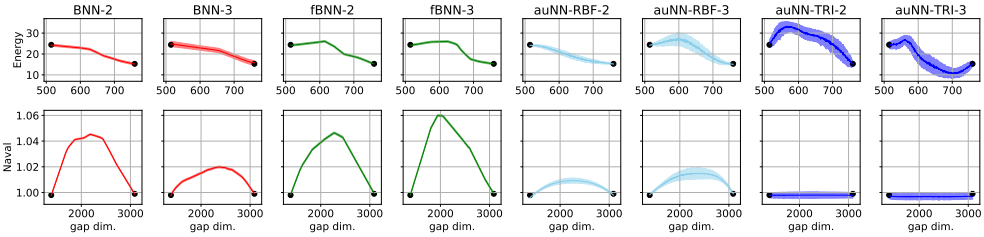

Figure 5: Predictive distribution (mean and one standard deviation) over a segment that crosses the gap, joining two training points from different connected components. auNN avoids overconfident predictions by allocating more uncertainty in the gap, where there is no observed data.

In addition to the paradigmatic cases of Energy and Naval illustrated here, four more datasets are included in Appendix C. Figure 7 there is analogous to Figure 4 here, and Tables 4 and 5 there show the full numeric results and ranks. We observe that auNN, specially through the triangular kernel, obtains the best results and does not fail catastrophically in any dataset (unlike BNN and fBNN, which do in Energy and Naval). Finally, the performance on the gap splits is complemented by that on standard splits, see Tables 6 and 7 in Appendix C. This shows that, in addition to the enhanced uncertainty estimation, auNN is a competitive alternative in general practice.

3.3 Comparison with DGPs

As explained in Section 2, the choice of a GP prior for activation stochasticity establishes a strong connection with DGPs. The main difference is that auNN performs a linear projection from D^{l-1} to D^l dimensions before applying D^l 1D GPs, whereas DGPs define D^l GPs directly on the D^{l-1} dimensional space. This means that auNN units are simpler than those of DGP, recall Figure 2a. Here we show two practical implications of this.

First, it is reasonable to hypothesise that DGP units may require a higher number of inducing points M than auNN, since they need to cover a multi-dimensional input space. By contrast, auNN may require a higher number of hidden units D, since these are simpler. Importantly, the computational cost is not symmetric in M and D, but significantly cheaper on D, recall Section 2. Figure 6b shows the performance of auNN and DGP for different values of M and D on the UCI Kin8 set (with one hidden layer; depth will be analyzed next). As expected, note the different influence by M and D: whereas auNN improves "by rows" (i.e. as D grows), DGP does it "by columns" (i.e. as M grows)[5]. Next section (Section 3.4), will show that this makes auNN faster than DGP in practice. An analogous figure for RMSE and full numeric results are in Appendix C (Figure 9 and Tables 9-10).

Second, auNN simpler units might be better suited for deeper architectures. Figure 6c shows the performance on the UCI Power dataset when depth is additionally considered. It can be observed that auNN is able to take greater advantage of depth, which translates into better overall performance.

[5]Interestingly, the fact that DGP is not greatly influenced by D could be appreciated in its recommended value in the original work (Salimbeni & Deisenroth, 2017). They set $D = \min(30, D^0)$, where D^0 is the input dimension. This limits D to a maximum value of 30.